

```

/**
 * User Identity and Persistence Model.
 *
 * Orchestrates the lifecycle of User entities within the local database.
 * This module manages cryptographic security (hashing/salting), account state
 * transitions, and complex multi-table lookups for roles and access levels.
 *
 * @module User-Persistence-Model
 * @copyright 2018-2026, Dennis Jorgenson
 */

"use strict";

import { ApiError } from "#db/types";
import type { IUser } from "#db/interfaces/user";

import { Update, Fetch, Key } from "#db/query.util";
import { hashKey } from "#lib/crypto.util";
import { createHmac } from "node:crypto";
import { hasValues, isEqual } from "#lib/std.util";

const module = "Auth";

/**
 * Generates a SHA-256 hash of a User object for identity validation or password hashing.
 *
 * @function hashPassword
 * @param {Partial<User.IUser>} props - The User object or properties to be hashed.
 * @returns {Uint8Array} A 32-byte binary Buffer (derived from the 64-character hex hash).
 *
 * @description
 * 1. Serializes the provided user properties into a JSON string.
 * 2. Computes a SHA-256 hash using the native Node.js `crypto` module.
 * 3. Iterates through the resulting 64-character hex string to convert it into a
 *    byte-aligned {@link Uint8Array}.
 * 4. Returns a {@link Buffer} instance, making it compatible with database binary fields.
 *
 * @example
 * const binaryHash = hashPassword({ email: "dennis@example.com", password: "..." });
 */

/**
 * DATABASE PASSWORD HASH (Internal Security)
 * Uses the local node's USER_SECRET to verify identity.
 */
export const hashPassword = (props: any): Buffer | undefined => {
  const { username, email, password } = props;

  // Use the .env secret as the primary pepper
  const salt = process.env.USER_SECRET;
  if (!salt || !password) return undefined;

  // Normalize to prevent "Djorgenson" vs "djorgenson" logic fails
  const identityKey = `${username.toLowerCase().trim()}:${email.toLowerCase().trim()}:${salt}`;

  // Create the HMAC Digest
  const digest = createHmac("sha256", identityKey).update(password).digest();

  // Returning a Buffer satisfies both Uint8Array requirements
  // and direct comparison logic in your Login function.
  return Buffer.from(digest);
};

/**
 * Generates cryptographic salts and hashes for a user password.
 *
 * Applies a 32-byte unique salt to the raw password input to generate
 * an encrypted Uint8Array for storage.
 *
 * @param props - Object containing username, email, and raw password string.
 * @returns Partial user object containing the new 32-byte hash and encrypted password.
 * @throws Error if props are undefined.
 */
export const SetPassword = async (props: Partial<IUser>): Promise<Partial<IUser>> => {
  if (!props) throw new ApiError(310, "Unauthorized access attempt; systems administrator notified.");

  const { username, email, password } = props;
  const hash = hashKey(32);
  const encrypt = hashPassword({ username, email, password, hash });

  return {
    hash,
    password: encrypt,
  };
};

/**
 * Updates a user's password to a temporary value and sets account state to 'Reset'.
 *
 * @param context - Log tracing context (defaults to "User").
 * @param credentials - User identification (username/email).
 * @param tempPass - The temporary raw password string to be hashed.
 * @param log - Logger instance for tracking the transaction.
 * @returns {Promise<boolean>} True if update succeeds, throws ApiError (302) if user not found.
 */
export const PasswordReset = async (credentials: Partial<IUser>, tempPass: string, context = module) => {
  context = `${context}.Password.Reset`;

  const exists = await Fetch<IUser>(credentials, { table: "vw_users" }, context);

  if (!exists) throw new ApiError(302, "Invalid user credentials");

  // Generate HMAC using the passed-in temporary password
  const newHash = hashPassword({ ...credentials, password: tempPass });
  const [{ user }] = exists;
  const reset = await Key<IUser, "user">({ status: "Reset" }, { table: `user`, context });

  await Update<IUser>({ user, password: newHash, state: reset }, { table: "user", keys: { user: "=" }, context });
  return true;
};

/**
 * Authenticates a user attempt against stored cryptographic hashes.
 *
 * Validates the provided credentials against the stored salt and encrypted
 * password. Checks for account status; accounts not marked 'Enabled' are
 * rejected with specific status codes.
 *
 * @param props - Login credentials including username/email and raw password.
 * @param log - Logger instance for auditing the login attempt.

```

```

* @returns {Promise<Partial<IUser>>}} User object with an error code (0: Success, 301: Disabled, 302: Invalid Credentials).
*/
export const Login = async (props: Partial<IUser>, context = module): Promise<Partial<IUser>> => {
  context = `${context}.Login`;
  if (!hasValues) throw new ApiError(310, "Unauthorized access attempt; systems administrator notified.");

  const user = await Fetch<IUser>({ username: props.username, email: props.email }, {table: `vw_users`, context});

  if (user) {
    const [login] = user;
    const { password, username, email, hash } = login;

    if (password instanceof Uint8Array) {
      const encrypt = Buffer.from(password);
      const key = hashPassword({ username, email, password: props.password, hash });

      if (isEqual(encrypt, key!)) {
        if (login.status === "Enabled") {
          Object.assign(login, { ...login, error: 0, message: "Connected" });
          return login;
        }
        Object.assign(login, { ...login, error: 301, message: `Your account is currently '${login.status}'. Contact your administrator.` });
        return login;
      }
      Object.assign(login, { ...login, error: 302, message: `Invalid username or password.` });
      return login;
    }
    Object.assign(login, { ...login, error: 311, message: `Internal error. Contact your administrator.` });
    return login;
  }

  const error: Partial<IUser> = {};
  Object.assign(error, { ...props, error: 302, message: `Invalid user credentials.` });
  return error;
};

```